

繪製網站：運用 Gemini 模型，發揮想像力打造網站！

來源：<https://codelabs.developers.google.com/draw-a-website-gemini?hl=zh-tw>

步驟數：9

在本程式碼研究室中，您將使用 Gemini 1.0 Pro Vision 和 Gemini 1.5 Pro 等多種產品，透過生成式 AI 模型，在短短幾分鐘內建構 Cloud Run 應用程式，將繪圖轉換為網站程式碼。

目錄

1. [1. 簡介](#)
2. [2. 事前準備](#)
3. [3. 步驟 1：啟動 Python Cloud Run 網頁應用程式](#)
4. [4. 步驟 2：建構前端](#)
5. [5. 步驟 3：建構後端 \(生成式 AI\)](#)
6. [6. 步驟 4：準備依附元件和 Dockerfile](#)
7. [7. 步驟 5：部署網路應用程式](#)
8. [8. 清除所用資源](#)
9. [9. 恭喜](#)

1. 簡介

在網頁開發領域，將設計概念轉化為實用網站的過程既耗時又複雜。不過，隨著 Gemini 等生成式 AI 模型問世，這個過程變得越來越簡化且容易上手。我們將建構專門的解決方案，將手繪線框圖轉換為網站程式碼。設計人員和開發人員都能運用這項強大工具，以前所未有的輕鬆和效率，實現網站願景。

在本實驗室中，我們將建構網頁應用程式，讓使用者透過 Vertex AI 的生成式 AI 模型 (Gemini 1.5 Flash、Gemini 1.5 Pro 等)，根據輸入的線框稿和提示生成網站程式碼 (HTML、CSS 和 JavaScript)。應用程式將使用熱門的 Python 網頁架構 Flask 建構，並使用 Vertex AI 用戶端程式庫與 Generative Models 服務互動。

建構項目

完成本實驗室後，您將擁有可從線框和提示生成圖片的網頁應用程式。您也會進一步瞭解如何使用 Vertex AI 的生成式 AI 模型。

您的網頁應用程式會如下所示：

Draw a Website

Choose File

No file chosen

Gemini 1.5 Flash

Convert this drawing into an html page

Submit

應用程式流程

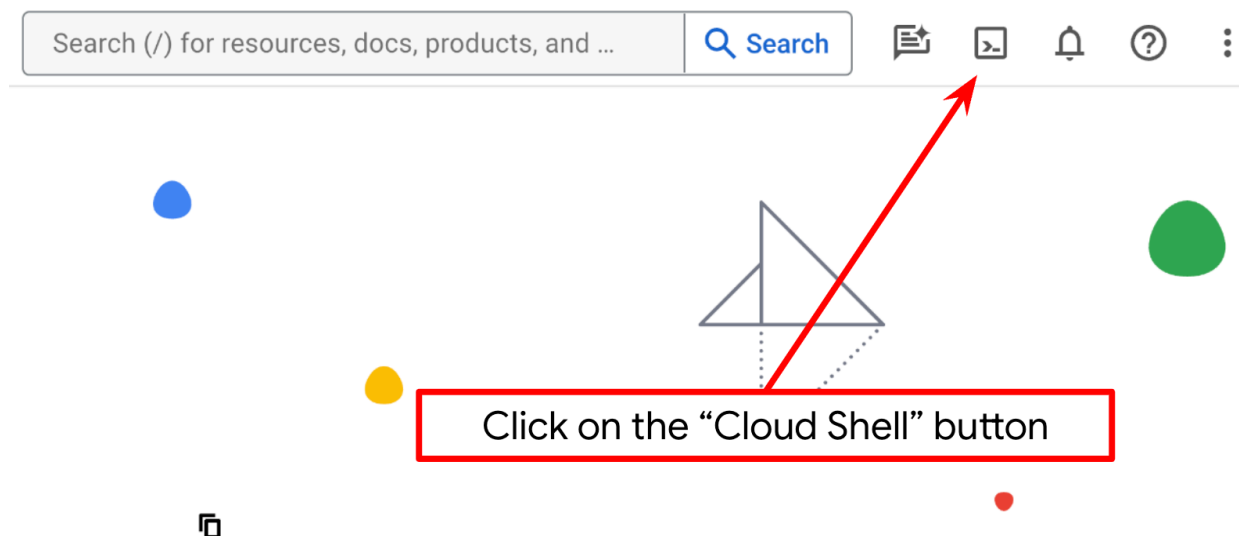
- 上傳手繪線框圖：**使用者可以將手繪線框圖的圖片上傳至應用程式。
- 選取模型：**應用程式提供一系列預先訓練的 Gemini 模型，這些模型經過最佳化，可產生不同設計風格的圖片。
- 提供提示詞：**使用者可以選擇提供文字提示詞，引導模型生成圖片。
- 生成網站程式碼：**應用程式會將線框稿和提示傳送給 Gemini，後者會生成對應的網站程式碼。
- 顯示結果：**生成的程式碼會顯示在應用程式的回覆頁面。

我們會先討論線框和提示的基本概念，以及如何使用這些工具產生網站程式碼。接著，我們將逐步說明如何建構網頁應用程式，包括如何處理使用者輸入內容、生成回覆及顯示結果。

步驟 2 · 約 0 分鐘

2. 事前準備

1. 在 [Google Cloud 控制台](#) 的專案選取器頁面中，選取或建立 Google Cloud [專案](#)。
2. 確保您的 Google Cloud 雲端專案有啟用計費服務。瞭解如何[檢查專案是否已啟用計費功能](#)。
3. 您將使用 [Cloud Shell](#)，這是 Google Cloud 中執行的指令列環境。如要存取，請按一下 Google Cloud 控制台頂端的「啟用 Cloud Shell」。



4. 連至 Cloud Shell 後，請使用下列指令確認驗證已完成，專案也已設為獲派的專案 ID：

```
gcloud auth list
```

5. 在 Cloud Shell 中執行下列指令，確認 gcloud 指令已瞭解您的專案。

```
gcloud config list project
```

6. 如果未設定專案，請使用下列指令來設定：

```
gcloud config set project <YOUR_PROJECT_ID>
```

7. 確認已啟用下列 API：

- Cloud Run
- Vertex AI

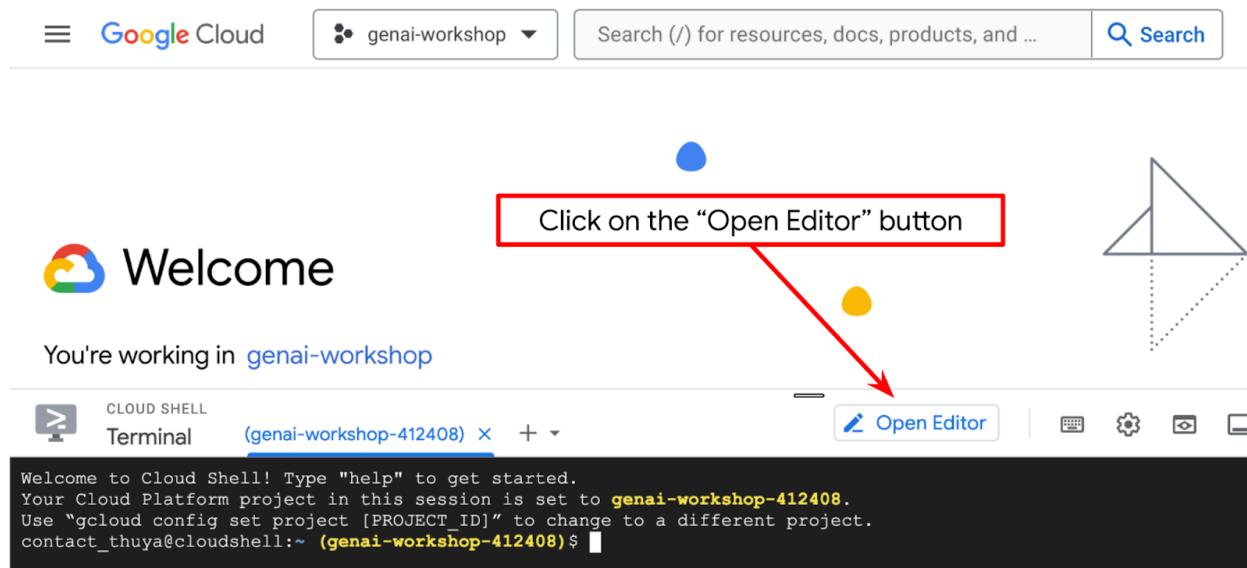
除了使用 gcloud 指令，您也可以透過這個[連結](#)前往控制台。如要瞭解 gcloud 指令和用法，請參閱[說明文件](#)。

步驟 3 · 約 0 分鐘

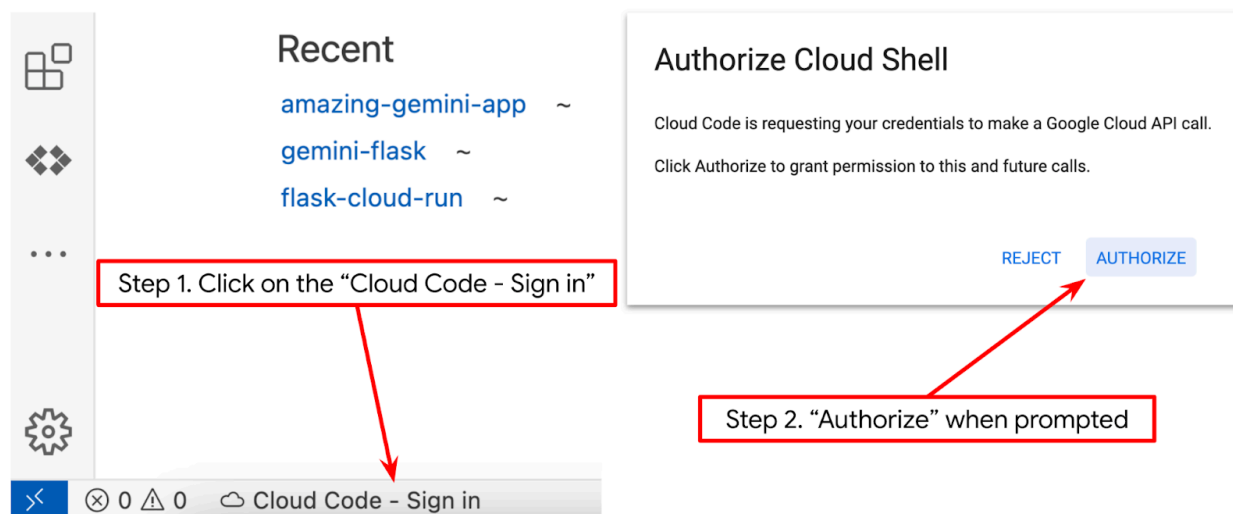
3. 步驟 1：啟動 Python Cloud Run 網頁應用程式

我們將先從 Cloud Shell 建立 Python [Cloud Run](#) 網頁應用程式範本。

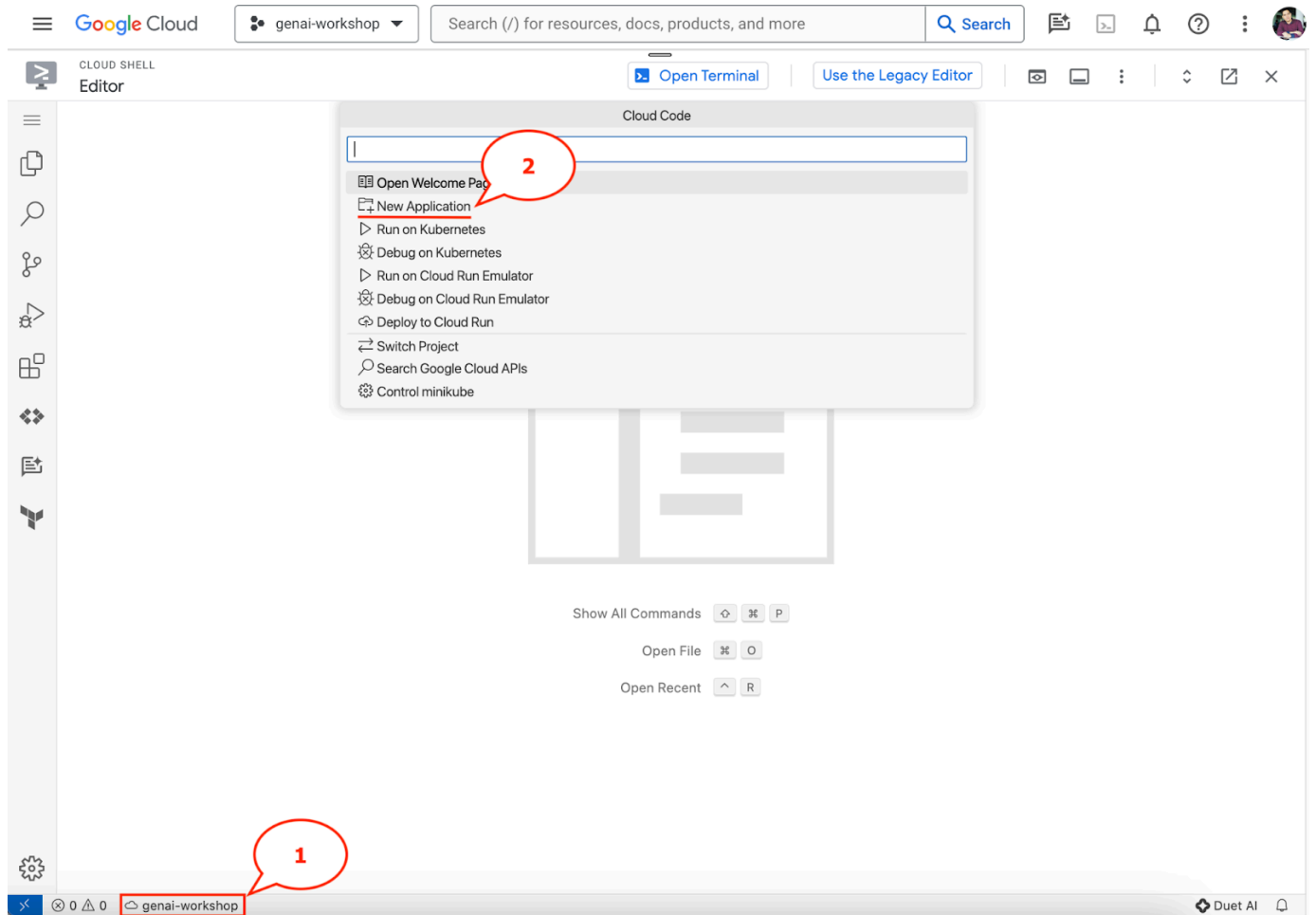
前往 [Cloud Shell 終端機](#)，然後點按「開啟編輯器」按鈕。



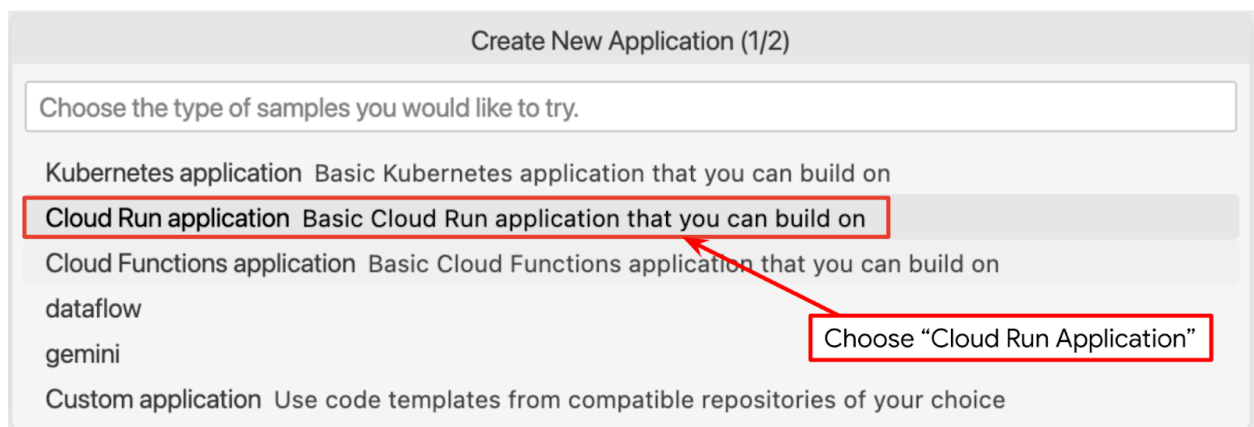
請確認 Cloud Shell 編輯器的左下角 (狀態列) 已設定 Cloud Code 專案，如下圖所示，且已設為啟用計費功能的有效 Google Cloud 專案。如果出現提示訊息，請點選「授權」。



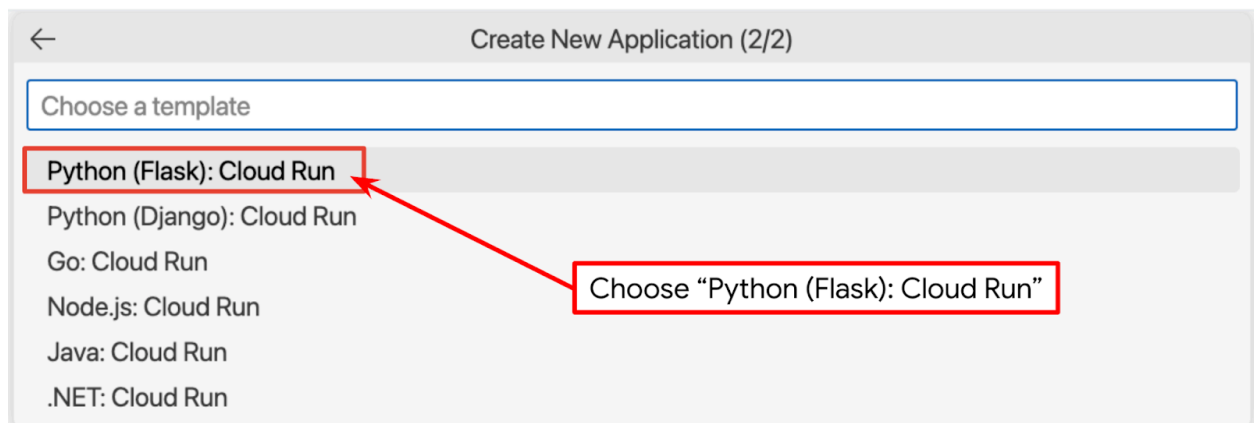
按一下狀態列上的現用專案，等待 Cloud Code 快顯視窗開啟。在彈出式視窗中選取「New Application」(新應用程式)。



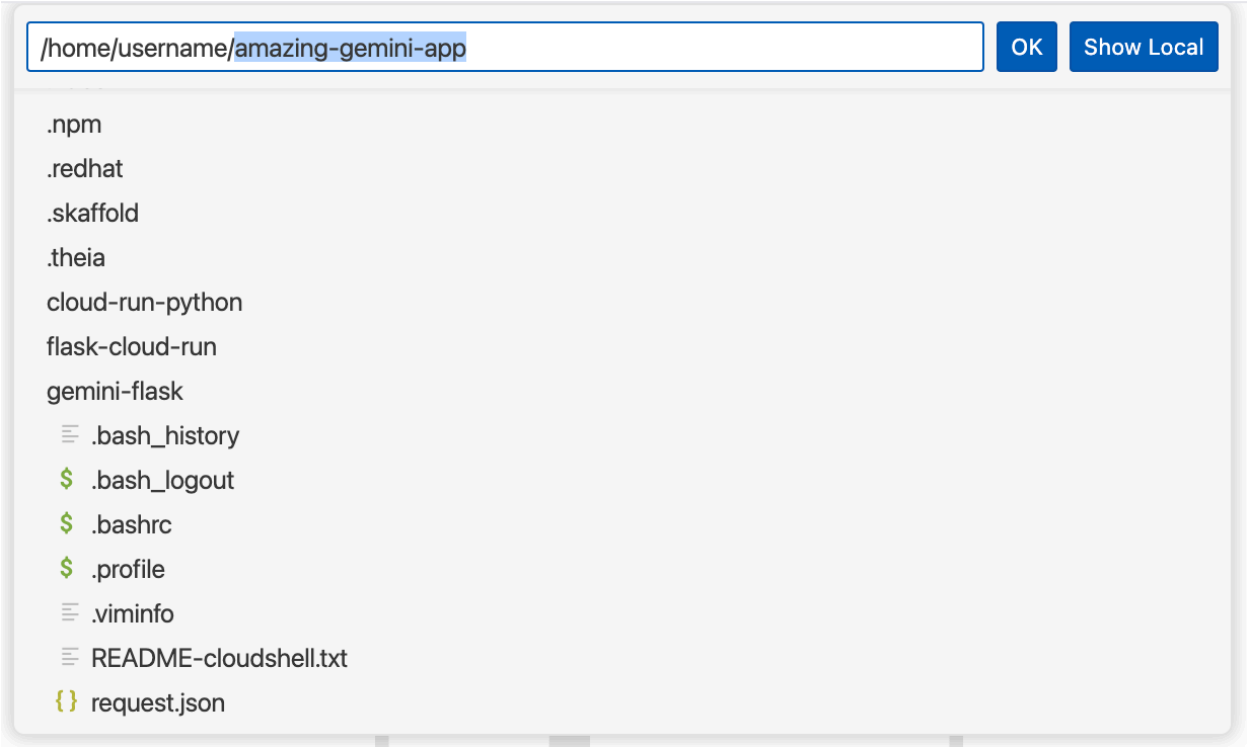
在應用程式清單中，選擇「Cloud Run Application」：



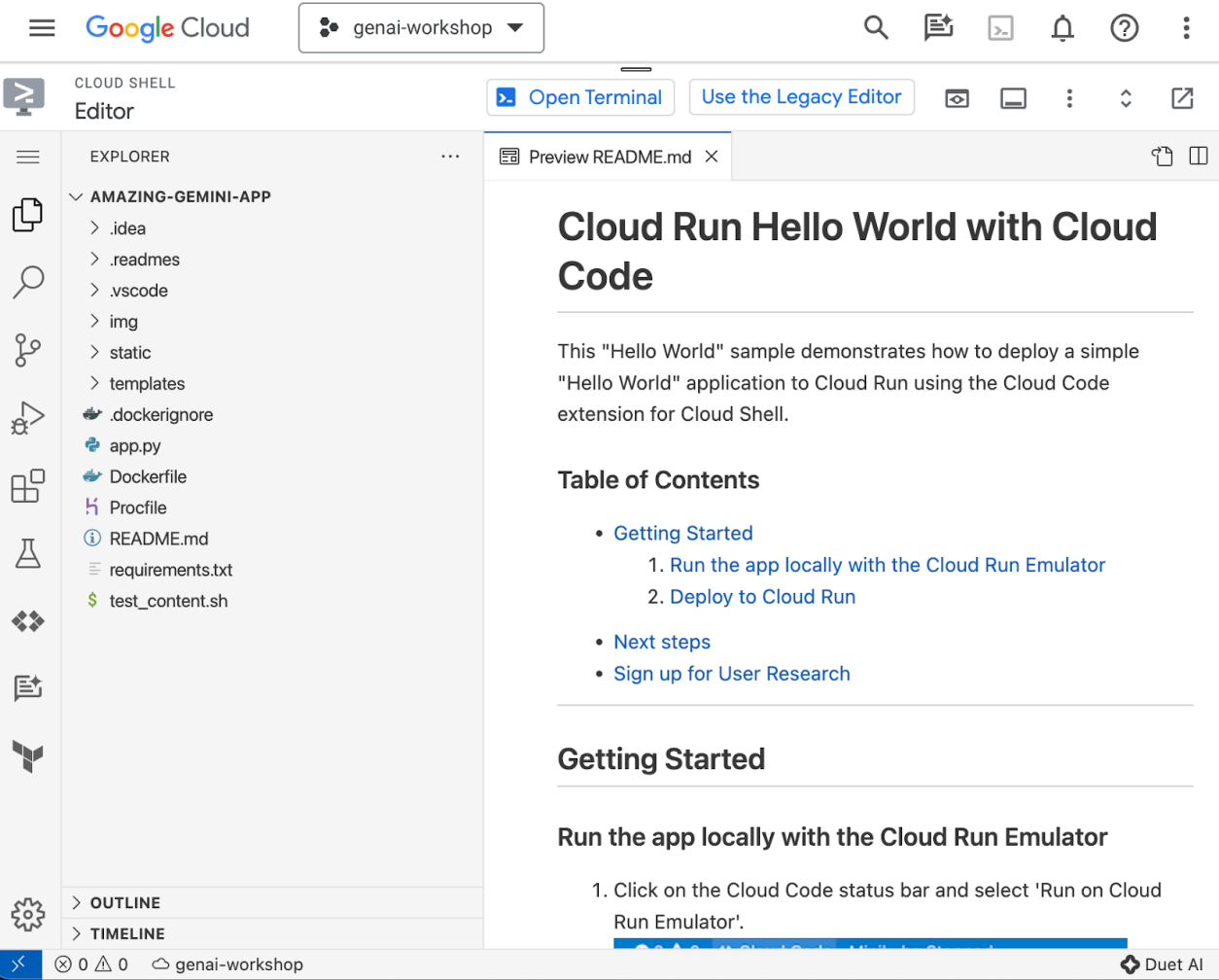
在第 2/2 頁，選取 Python Flask 範本：



提供您想要的專案名稱 (例如「amazing-gemini-app」)，然後按一下「OK」：



系統隨即會開啟您剛設定的新專案範本。



使用 Google Cloud Shell 啟動網頁應用程式就是這麼簡單。

步驟 4 · 約 0 分鐘

4. 步驟 2：建構前端

為此，我們需要 HTML 網頁。該頁面會包含定義網頁應用程式使用者介面的程式碼。使用者可透過表單上傳手繪線框稿圖像、選取生成模型，並提供文字提示詞。提交表單後，結果會顯示在另一個分頁中。

複製下列程式碼，並取代 **templates** 資料夾中的 index.html 檔案：

```

<!DOCTYPE html>
<html>
<head>
  <title>Draw a Website</title>
  <style>
    body {
      font-family: sans-serif;
      display: flex;
      justify-content: center;
      align-items: center;
      min-height: 100vh; /* Ensure form is centered vertically */
      background-color: #f4f4f4;
    }
    .container {
      background-color: white;
      padding: 30px;
      border-radius: 8px;
      box-shadow: 0 2px 5px rgba(0, 0, 0, 0.1);
      text-align: center;
    }
    h2 {
      text-align: center;
      margin-bottom: 20px;
    }
    input[type="file"], textarea, select {
      width: 100%;
      padding: 10px;
      margin-bottom: 15px;
      border: 1px solid #ccc;
      border-radius: 4px;
      box-sizing: border-box;
    }
    button {
      background-color: #4CAF50;
      color: white;
      padding: 12px 20px;
      border: none;
      border-radius: 4px;
      cursor: pointer;
    }
  </style>
</head>
<body>
  <div class="container">
    <h2>Draw a Website</h2>
    <form action="/response" target="_blank" method="post" enctype="multipart/form-data">
      <input type="file" id="image-upload" name="image-upload" accept=".png, .jpg,
.jpeg">
      <select name="model">
        <option value="gemini-1.5-flash-001">Gemini 1.5 Flash</option>
        <option value="gemini-1.5-pro-001">Gemini 1.5 Pro</option>
        <option value="gemini-1.0-pro-vision-001">Gemini 1.0 Pro Vision</option>
      </select>
    </form>
  </div>
</body>
</html>

```

```
        </select>
        <textarea name="prompt" placeholder="Write your prompt here. For example: 'Convert
this drawing into an html page'">Convert this drawing into an html page</textarea>
        <button type="submit">Submit</button>
    </form>
</div>
</body>
</html>
```

使用者與應用程式互動時，會發生下列動作：

1. 使用者選取線框圖、模型，然後輸入提示。
2. 使用者點選「提交」按鈕後，系統會使用 HTTP POST 方法，將表單資料 (圖片、模型和提示) 傳送至 `/response` 網址。
3. 伺服器端程式碼 (在 `app.py` 中實作) 會處理表單資料，並使用指定的模型和提示詞生成回覆。
4. 生成的回覆會顯示在新分頁中。

我們現在已準備好網頁應用程式的前端部分。

步驟 5 · 約 0 分鐘

5. 步驟 3：建構後端 (生成式 AI)

現在來編寫這個網頁應用程式的主要部分。app.py 檔案會接收使用者輸入內容的圖片、模型選擇和提示，並將其轉換為網站程式碼。

複製 app.py 的完整程式碼：

```

# Import the necessary libraries.
import os
import random
from flask import (
    Flask,
    render_template,
    request,
    redirect
)

import vertexai
from vertexai.generative_models import (
    GenerativeModel,
    Image
)

# Initialize the Flask app.
app = Flask(__name__)
app.config['MAX_CONTENT_LENGTH'] = 16 * 1024 * 1024 # 16 MB per image

# TODO: Replace "YOUR_PROJECT_ID" before running
# Initialize the Vertex AI client library.
vertexai.init(project="YOUR_PROJECT_ID", location="us-central1")

# Define a function to generate response from a wireframe and a prompt.
def generate(wireframe, model, prompt):
    '''Generates a response from a wireframe and a prompt.
    Args:
    wireframe: The wireframe image.
    model: The generative model to use.
    prompt: The prompt to use.
    Returns: The generated response.
    '''
    # Create a GenerativeModel object.
    model = GenerativeModel(model)

    # Create a list of contents to pass to the model.
    contents = [
        wireframe,
        prompt
    ]

    # Generate the response.
    responses = model.generate_content(
        contents=contents,
        stream=True,
    )

    # Concatenate the response text.
    response = ""
    for res in responses:
        response += res.text.strip()

```

```

# Return the generated response.
return response

# Define the home page route.
@app.route('/', methods=['GET'])
def index():
    '''Renders the home page.
    Returns:The rendered template.
    '''
    return render_template('index.html')

# Define the response route.
@app.route('/response', methods=['GET', 'POST'])
def response():
    '''Handles the response to the user's input.
    Returns:The rendered template.
    '''
    # If the request is a POST request, process the form data.
    if request.method == 'POST':
        # Get the uploaded image from the request.
        uploaded_image = request.files['image-upload']

        # Convert the uploaded image to a wireframe image.
        wireframe = Image.from_bytes(uploaded_image.read())

        # Get the model and prompt from the request.
        model = request.form['model']
        prompt = request.form['prompt']

        # Generate the response and render the response.
        try:
            response = generate(wireframe, model, prompt)
            response = response.replace("`html", "").replace("`", "").strip()
            return response
        except ValueError as e:
            raise e

    # If the request is a GET request, redirect to the home page.
    else:
        return redirect('/')

# Run the app.
if __name__ == '__main__':
    # Get the server port from the environment variables.
    server_port = os.environ.get('PORT', '8080')

    # Run the app.
    app.run(debug=False, port=server_port, host='0.0.0.0')

```

程式碼的本質如下：

1. 這段程式碼會匯入應用程式所需的程式庫：

Flask：適用於 Python 的輕量型網頁架構。

os：用於與作業系統互動。

random：用於產生隨機數字。

vertexai：Vertex AI 用戶端程式庫。

GenerativeModel 和 Image：Vertex AI Generative Models 程式庫中的類別。

2. 初始化 Flask 應用程式：

接著，它會初始化 Flask 應用程式，並將上傳圖片的大小上限設為 16 MB。

3. 初始化 Vertex AI 用戶端

這會使用指定的專案 ID 和位置初始化 Vertex AI 用戶端程式庫。請務必將 YOUR_PROJECT_ID 替換為您的專案 ID。

4. 定義 generate 函式

這個函式會將線框圖、生成模型和提示詞做為輸入內容。接著，系統會使用指定的模型和提示詞產生網站 HTML。系統會以字串形式傳回生成的回覆。

5. 定義首頁路徑

這個函式會定義首頁路徑。當使用者造訪應用程式的根網址時，系統會呼叫這個函式。這個函式會轉譯 index.html 範本，也就是應用程式的首頁。

6. 定義回應路徑

這個函式會定義回應路徑。使用者在首頁提交表單時，系統會呼叫這個函式。系統會處理上傳的圖片、模型和提示，然後生成網站程式碼。生成的回覆會顯示在新分頁中。

7. 執行應用程式

這部分程式碼會檢查指令碼是否以主要程式的形式執行。如果是，則會從環境變數取得伺服器通訊埠，並在指定的通訊埠上執行應用程式。

步驟 6 · 約 0 分鐘

6. 步驟 4：準備依附元件和 Dockerfile

確認 requirements.txt 檔案中包含下列依附元件：

```
Flask==2.3.3
requests==2.31.0
debugpy # Required for debugging.
google-cloud-aiplatform>=1.38
```

將 Dockerfile 內容替換為下列內容：

```
# Python image to use.
FROM python:3.11-slim

# Set the working directory to /app
WORKDIR /app

# copy the requirements file used for dependencies
COPY requirements.txt .

# Install any needed packages specified in requirements.txt
RUN pip install -r requirements.txt

# Copy the rest of the working directory contents into the container at /app
COPY . .

# Run app.py when the container launches
ENTRYPOINT ["python", "app.py"]
```

步驟 7 · 約 0 分鐘

7. 步驟 5：部署網路應用程式

建立應用程式元件後，我們來部署應用程式。

前往 [Cloud Shell 終端機](#)，確認目前專案已設為有效專案。如果不是，請使用 `gcloud config set project [PROJECT_ID]` 指令設定專案 ID：

```
gcloud config set project [PROJECT_ID]
```

然後依序輸入下列指令：

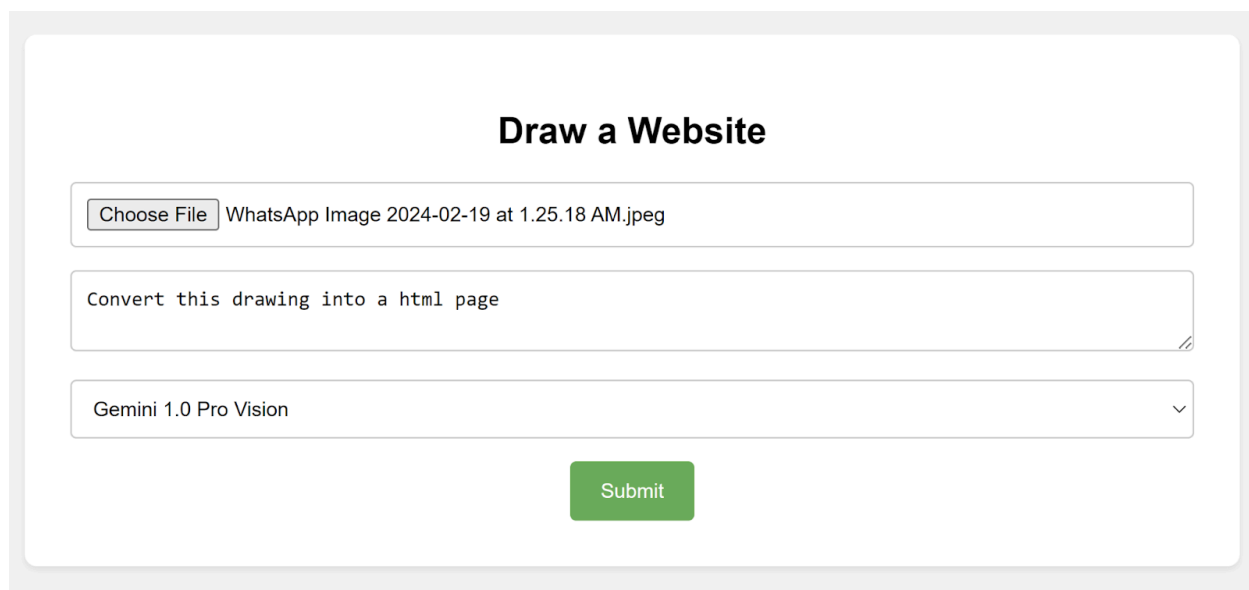
```
cd draw-a-website  
gcloud run deploy --source .
```

系統會提示您輸入服務名稱，假設為「**draw-website**」。選擇「us-central1」地區的對應編號。系統詢問是否要允許**未經驗證的叫用要求**時，請說「**y**」。請注意，我們在這裡允許未經驗證的存取要求，因為這是示範應用程式。建議您為企業和正式版應用程式使用適當的驗證方式。

部署完成後，您會取得類似下方的連結：

****https://draw-website-*****eua-uc.a.run.app/**

請測試應用程式：



The screenshot shows a web application titled "Draw a Website". It features a file upload section with a "Choose File" button and a selected file named "WhatsApp Image 2024-02-19 at 1.25.18 AM.jpeg". Below this is a text input field containing the prompt "Convert this drawing into a html page". At the bottom, there is a dropdown menu showing "Gemini 1.0 Pro Vision" and a green "Submit" button.

8. 清除所用資源

如要避免系統向您的 Google Cloud 帳戶收取本程式碼研究室所用資源的費用，請按照下列步驟操作：

1. 在 Google Cloud 控制台中前往「管理資源」頁面。
 2. 在專案清單中選取要刪除的專案，然後點按「刪除」。
 3. 在對話方塊中輸入專案 ID，然後按一下「Shut down」(關機) 即可刪除專案。
 4. 或者，您也可以前往控制台的「Cloud Run」，選取剛部署的服務並刪除。
-

步驟 9 · 約 0 分鐘

9. 恭喜

恭喜！您已成功在 Cloud Run 上部署以 Python Flask 建構的快速網頁應用程式，可將手繪稿轉換為網站。完整存放區請參閱[這裡](#)。「draw-a-website」應用程式展現了 Gemini 的強大功能，可簡化網頁開發程序。運用 AI 技術，設計人員和開發人員就能更快速、準確且富含創意地建立網站。隨著生成式 AI 模型持續演進，我們可以預期未來會出現更多具突破性的應用程式。
